Prashna-AI Platform Presentation Script

> ***Presenter Check-in:*** *Take a deep breath! This script is designed to help you confidently show off the amazing AI features you've built into your app. Read the* ***bold outline boxes*** *aloud, and follow the* `highlighted italic text` *for actions to take on screen.*

1. The Landing Page

[Start on the Landing Page. Toggle the dark/light mode switch a few times to show off the UI.]

"Hello everyone. Today I am excited to present Prashna-AI, an intelligent, adaptive quiz engine that I've been developing. The goal of this platform is to use advanced Machine Learning and AI to convert static study materials into dynamic, personalized learning experiences."

"As you can see, the platform features a modern, responsive design with a built-in dark and light mode toggle for better accessibility."

2. Registration & Onboarding

[Click 'Get Started' and navigate to the Registration page.]

"When a new user joins, they don't just create an account—they set up an AI profile. During registration, users select their preferred baseline difficulty, subjects of interest, and the types of questions they like to answer. This seeds our adaptive learning algorithm."

[Log into your existing developer account.]

3. The Dashboard

[You are now on the Dashboard.]

"Welcome to the central Dashboard. At the very top, you'll see my current 'Elo Rating.' Similar to chess ranking systems, Prashna-AI constantly analyzes my performance and response times to evaluate my true skill level in real-time."

"The Dashboard also provides instant feedback on my overall accuracy, and uses pattern analysis to highlight my 'Weak Areas' and provide personalized study recommendations."

4. Content Upload (The ML Engine at Work)

```
[Click on 'Upload Content' from the sidebar]
```

> "To generate a quiz, we first need context. Users can upload raw text, fetch content directly from a URL, or upload PDF documents. Let me paste some text here."

```
[Paste some educational text into the box and click 'Upload & Analyze']
```

> "Right now, the backend is running a complex NLP (Natural Language Processing) pipeline. It analyzes the text complexity, chunks it into semantic blocks, uses Named Entity Recognition to index key concepts, and even runs the content through an AI summarizer."

```
[Show the upload success screen]
```

> "Here you can see the results: a complexity score, an AI-generated TL;DR summary, and the core topics automatically discovered by our clustering algorithm."

5. Quiz Generation & Adaptive Mode

```
[Click 'Generate Quiz from this Content']
```

> "Now we move to the Quiz Generator. Here, I can select what types of questions I want: Multiple Choice, True/False, Fill in the Blank, or Short Answer."

> "But the standout feature here is the 'Adaptive Mode' toggle. When this is ON, the system ignores manual difficulty settings. Instead, it reads my exact Elo rating, dynamically assigns a difficulty tier, and instructs the AI to generate questions according to Bloom's Taxonomy. For example, lower Elo gets 'Recall' questions, while higher Elo gets 'Analyze and Evaluate' questions."

```
[Click 'Generate Quiz' and wait for it to load.]
```

6. Taking the Quiz

```
[You are now on the active Quiz page. Answer a few questions.]
```

> "While taking the quiz, the AI provides instant feedback. For multiple-choice questions, it uses semantic similarity models to generate highly plausible 'distractor' answers, preventing users from just guessing."

```
[Type a short answer to a text question.]
```

> "For written short answers, we built a lenient, tiered grading algorithm. It doesn't look for an exact word-for-word string match. Instead, it measures keyword intersections and substrings, meaning if I write the core concept correctly, the AI is smart enough to mark it as correct!"

```
[Click Submit]
```

7. Results & Real-time Evaluation

```
[You are now on the Results page]
```

> "Because we utilized asynchronous background tasks on the backend, quiz submission and processing happens instantly—in less than a second."

> "Here on the results page, you can see exactly how this specific session impacted my Elo rating. My Elo goes up for correct answers, down for incorrect ones, and is weighted by the inherent difficulty of the questions I was served."

8. Advanced Analytics & Knowledge Graph

```
[Click on 'Analytics' in the sidebar]
```

> "To truly track progress, we built an advanced Analytics suite. It tracks topics over time, and uses historical data to actually predict my future performance accuracy on specific subjects."

```
[Scroll down to the Knowledge Graph or Topic Clusters if data is populated]
```

> "We even render a visual 'Knowledge Graph' mapping the relationships between concepts I've learned, and a topic cluster scatter-plot powered by dimensionality reduction algorithms like PCA (Principal Component Analysis)."

9. Admin Panel (Role-Based Access)

```
[Click on 'Admin Panel' in the sidebar]
```

> "Finally, as the developer, I have access to a secure, role-based Admin Panel. This page is strictly locked down; the backend verifies the user's email token on every request, ensuring only my specific email can view this page."

> "Here, I can monitor high-level platform analytics, review flagged questions that users reported, and analyze system-wide performance, all powered by highly-optimized bulk database queries to ensure the page loads instantly even with thousands of rows."

> "That concludes the tour of Prashna-AI. By combining React, FastAPI, Supabase, and advanced AI models, we've built a truly intelligent platform that adapts to the user, rather than forcing the user to adapt to the platform. Thank you!"

Appendix A: Project Information (What we've built)

Overview: Prashna-AI is an adaptive, AI-driven learning platform designed to generate dynamic quizzes from static uploaded content (text, PDFs, or URLs). The core objective is to move away from static testing and instead test students adaptively, matching the difficulty of the questions to their proven skill level using an Elo rating system.

Key Achievements:

- Developed a fully functioning full-stack application from scratch.
- Integrated a real-time Elo rating system to track user skill.
- Built a multi-provider fallback AI chain (OpenAI -> Groq -> Ollama) to ensure high uptime for quiz generation.
- Implemented an intelligent content processing pipeline that chunks data, analyzes complexity, discovers topics (via NLP), and summarizes.
- Handled advanced UI state management, real-time feedback, and secure, role-based admin routing.
- Optimized massive database retrievals, eliminating N+1 query loops to make the dashboard load instantly.

Appendix B: Feature Breakdown

Frontend Features (React + Vite)

- **Responsive, Modern UI:** Clean, animated interface with a seamless drag-and-drop file upload, custom toggle switches, and a global Light/Dark mode.
- **Adaptive Quiz Interface:** Real-time progress bars, interactive option highlighting, and a lenient short-answer grader that understands context rather than punishing typos.
- **Dynamic Dashboard & Analytics:** Renders visual data including Elo tracking, Weak Area tagging, and performance prediction.
- **Role-Based Access Control (RBAC):** Secure routing that completely hides and blocks the Admin Panel from non-developer accounts.

Backend Features (FastAPI + Python)

- **Advanced NLP Pipeline:** Uses `spaCy` to extract entities and text complexity (`textstat`), and `BERTopic` / frequency analysis to discover core concepts in uploaded text.
- **Adaptive Engine:** Calculates standard Elo Rating updates instantly after every quiz, tracking historical correctness and response times.
- **Multi-Agent LLM Generation:** Sends targeted prompts using specific cognitive levels from Bloom's Taxonomy to generate tailored questions. Uses background tasks to prevent the server from hanging during intense processing.
- **Supabase Integration & Optimization:** Handles authentication natively, and uses precisely tailored bulk SQL/REST queries to fetch hundreds of attempts in a fraction of a second without event-loop blocking.

Appendix C: Potential Q&A with Your Coordinator

Q: How does the Adaptive Mode actually work?

A: "When enabled, the system reads the user's current Elo Rating. If the Elo is low (<800), it forces the AI to generate 'Easy' questions focused on basic recall. If it's high (>1200), it forces 'Hard' questions focused on analysis and evaluation, based on Bloom's Taxonomy. The middle tier gets 'Medium' questions."

Q: Why did you choose FastAPI for the backend?

A: "FastAPI is incredibly fast because it's built on Starlette and Pydantic. It natively supports asynchronous programming, which is crucial for an AI app where we spend a lot of time waiting for LLM network responses. It prevents the server from blocking other users."

Q: How are you managing the AI generation if the API goes down or is slow?

A: "We implemented a Multi-Provider Fallback Chain. The system tries OpenAI first. If it times out or fails, it automatically falls back to Groq (which is exceptionally fast), and can even fall back to a local Ollama instance if necessary, ensuring constant uptime."

Q: How do you handle the grading for Short Answer text questions?

A: "We implemented a lenient string-matching algorithm. Instead of demanding an exact character-to-character match, the grader splits both the correct answer and the user's answer into core keywords (ignoring stop words). If there is a sufficient intersection of keywords or a substring match, it marks the answer as correct. It understands the concept rather than just the exact spelling."

Q: Did you run into any major performance issues, and how did you solve them?

A: "Yes, initially the Dashboard took very long to load because of an 'N+1 Query Loop'—the backend was pinging the database separately for every single quiz a user had ever taken. I refactored the data layer to fetch all attempts in a single massive bulk query, reducing the load time to under a second. I also had to convert async routes to standard thread-based routes to allow FastAPI to handle synchronous database calls efficiently."

Q: How is the Admin Panel secured?

A: "It uses strict Role-Based Access Control both on the frontend and backend. The React frontend will bounce any unauthorized user back to the dashboard if they manually enter the URL. Even if they bypass the UI and hit the API directly, the FastAPI backend checks the JWT token's email payload and strictly returns a '403 Forbidden' error if the email doesn't manually match the developer account."

Q: What APIs did you use in this project and how do they work?

A: "I used three primary third-party APIs:

1. The Supabase API: This acts as our Backend-as-a-Service. Instead of writing raw SQL, our internal Python FastAPI calls the Supabase API to securely handle user authentication, insert new users, and store/retrieve all of the generated quizzes and relational data through REST endpoints.

2. The Groq Cloud API: We use this as our primary inference engine for Artificial Intelligence. I POST the extracted study material and a tailored prompt to the Groq API endpoint. Groq runs an open-source model (like LLaMA-3) on incredibly fast LPU hardware and returns the generated quiz format disguised as raw JSON data, which I then parse into Python objects.

3. The OpenAI API: This serves as both an alternative generator and our summarization engine. We send it text, and it returns high-level summaries and key-takeaways using GPT-4o."

Styled Markdown Example

This is a paragraph with **bold** and *italic* text.

Lists Example

- First item
- Second item with **bold**
- Nested item with *italic*
- Another nested item
- Third item

Java Code Example

```
function greeting(name) {
return `Hello, ${name}!`;
}
```

Python Example

```
def fibonacci(n):
if n <= 1:
    return n
else:
    a, b = 0, 1
    for _ in range(n - 1):
        a, b = b, a + b
    return b

# List comprehension example
squares = [x**2 for x in range(10)]
```

Blockquote Example

> This is a blockquote.
> It can span multiple lines.

Table Example

Feature	Description
Tables	Organized data display
Lists	Bullet points and numbers
Code	Syntax highlighted blocks

Link Example

Visit GitHub

Cat Image Example